Experiment 3- Implementation of concurrent day-time client-server application

Server-Side Code Summary

1. Create and Bind Server: Instantiate a `ServerSocket` on a specific port (e.g., 5000).

2. Listen and Accept Connections: Enter loop to listen for and accept incoming client connections.

3. Handle Client Request: For each client, retrieve the client's IP address and send the current date and time.

4. Display the client’s IP address in the server console.

5. Manage Resources: Close the output stream and client socket after sending the response.

6. Shutdown: Close the server socket when shutting down the server.

Client-Side Code Summary

1. Create and Connect Client Socket: Create a `Socket` and connect it to the server's hostname and port.

2. Send Client's IP Address: Retrieve the client’s own IP address and send it to the server.

3. Receive and Display Server Response: Read the server’s response (date and time) and display it.

4. Manage Resources: Close the input/output streams and the client socket after the interaction.

5. Shutdown: End the client program after the connection is properly closed.

Server

1. Create a server socket and bind it to the port.
2. Listen for new connections and when a connection arrives, accept it.
3. Send server's date and time to the client.
4. Read the client's IP address sent by the client.
5. Display the client details.
6. Repeat steps 2-5 until the server is terminated.
7. Close all streams.
8. Close the server socket.
9. Stop.

Client

1. Create a client socket and connect it to the server's port number.
2. Retrieve its own IP address using built-in function.
3. Send its address to the server.
4. Display the date & time sent by the server.
5. Close the input and output streams.
6. Close the client socket.
7. Stop.

Server_DT

```
import java.net.*;
import java.io.*;
import java.util.Date;
public class Server_DT {

    public static void main(String[] args)throws IOException {
        // TODO code application logic here

        //Step 1. Reserve a port number on the Server to offer this service
        ServerSocket ss= new ServerSocket(5000);
        //(Optional)To confirm Server Reserved specified port or not
        System.out.println("The Server has reserved port No.: "+ss.getLocalPort()+" for this Service");

        //Step 2. Now create a Client Socket on Server for Bidirectonal Communication.
        //Socket is created only when client communicates with the server
```

```
Socket cs=ss.accept();

//To confirm Server communicated through the socket or not
 System.out.println("Client with IP Address "+cs.getInetAddress()+" has communicated via port
        No.: "+cs.getPort());

Date d=new Date();
String s="Current Date & Time on Server is:"+d;

//Send String s to client via client socket
PrintWriter toclient=new PrintWriter(cs.getOutputStream(),true);
toclient.print(s);
toclient.close();
cs.close();
ss.close();
}
```

Client_DT

```
import java.net.*;
import java.io.*;

public class Client_DT {

   public static void main(String[] args) throws UnknownHostException,IOException {
      // TODO code application logic here
      //Step 1. Create a client socket to connect to Server
      Socket cs= new Socket("LocalHost",5000);

      //To confirm Client is communicating through the port
      System.out.println("Client "+cs.getInetAddress()+" is communicating from port No.:
            "+cs.getPort());

      //Receive Date Sent by Server
```

```
        BufferedReader fromserver=new BufferedReader(new InputStreamReader(cs.getInputStream()));


        System.out.println(fromserver.readLine());
        fromserver.close();
        cs.close();


    }
}
```